

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»
Институт информатики и кибернетики

Отчет по лабораторной работе №7

Дисциплина: «ООП»

Тема «Реализация паттернов и рефлексии»

Выполнила: Степанова А. Д.

Группа: 6201-120303

Самара, 2025

Задание на лабораторную работу

Внести изменения в существующий набор типов табулированных функций, позволяющие обрабатывать точки функций по порядку (паттерн «Итератор»), а также выбирать тип объекта табулированной функции при его неявном создании (паттерн «Фабричный метод» и средства рефлексии).

Задание 1

Сделала так, чтобы все объекты типа TabulatedFunction можно было использовать в качестве объекта-агрегата в for-each, извлекаемые объекты имеют тип FunctionPoint.

```
public interface TabulatedFunction extends Function, Cloneable,
Iterable<FunctionPoint> {...}
```

Реализовала итератор в ArrayTabulatedFunction.

```
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private int index = 0; 2 usages

        public boolean hasNext() {
            return index < pointsCount;
        }

        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("Нет следующего элемента");
            }

            return new FunctionPoint(points[index++]);
        }

        public void remove() {
            throw new UnsupportedOperationException("Удаление не доступно");
        }
    };
}
```

Реализовала итератор в LinkedListTabulatedFunction.

```

public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private FunctionNode current = head.next; 3 usages
        private int currentIndex = 0; 2 usages

        public boolean hasNext() {
            return currentIndex < pointsCount;
        }

        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("Нет следующего элемента");
            }
            FunctionPoint point = new FunctionPoint(current.point);
            current = current.next;
            currentIndex++;
            return point;
        }

        public void remove() {
            throw new UnsupportedOperationException("Удаление не доступно");
        }
    };
}

```

Протестировала итераторы.

```

import functions.*;
import functions.basic.*;

public class Main {
    public static void main(String[] args) {

        //1
        System.out.println("\nТест итератора");

        FunctionPoint[] points = {
            new FunctionPoint(x: 3, y: 9),
            new FunctionPoint(x: 5, y: 25),
            new FunctionPoint(x: 7.5, y: 56.25)
        };

        ArrayTabulatedFunction arrayFunction = new ArrayTabulatedFunction(points);
        LinkedListTabulatedFunction listFunction = new LinkedListTabulatedFunction(points);

        System.out.println("Итерация по ArrayTabulatedFunction:");
        for (FunctionPoint p : arrayFunction) {
            System.out.println("  " + p);
        }

        System.out.println("\nИтерация по LinkedListTabulatedFunction:");
        for (FunctionPoint p : listFunction) {
            System.out.println("  " + p);
        }
    }
}

```

```
Тест итератора
Итерация по ArrayTabulatedFunction:
(3,00; 9,00)
(5,00; 25,00)
(7,50; 56,25)

Итерация по LinkedListTabulatedFunction:
(3,00; 9,00)
(5,00; 25,00)
(7,50; 56,25)
```

Задание 2

В пакете functions описала интерфейс фабрик табулированных функций TabulatedFunctionFactory.

```
package functions;

public interface TabulatedFunctionFactory { 4 usages 2 implementations
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount);
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values);
    TabulatedFunction createTabulatedFunction(FunctionPoint[] points); 1 usage 2 implementations
}
```

В класс ArrayTabulatedFunction добавила вложенный класс ArrayTabulatedFunctionFactory.

```
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory { 1 usage

    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) { 1 usage
        return new ArrayTabulatedFunction(points);
    }
}
```

В класс LinkedListTabulatedFunction добавила вложенный класс LinkedListTabulatedFunctionFactory.

```

public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory { 2 usages
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }

    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) { 1 usage
        return new LinkedListTabulatedFunction(points);
    }
}

```

В классе TabulatedFunctions объявила приватное статическое поле типа TabulatedFunctionFactory и метод setTabulatedFunctionFactory(), позволяющий заменить объект фабрики.

```

private static TabulatedFunctionFactory factory = new 4 usages
    LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory();

public static void setTabulatedFunctionFactory(TabulatedFunctionFactory factory) {
    TabulatedFunctions.factory = factory;
}

```

Описала три перегруженных метода TabulatedFunction createTabulatedFunction(), возвращающих объекты табулированных функций, созданные с помощью текущей фабрики.

```

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
    return factory.createTabulatedFunction(leftX, rightX, pointsCount);
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
    return factory.createTabulatedFunction(leftX, rightX, values);
}

public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) { 2 usages
    return factory.createTabulatedFunction(points);
}

```

В методах tabulate, inputTabulatedFunction, readTabulatedFunction заменила явное создание объектов табулированных функций с помощью конструкторов на вызов метода createTabulatedFunction().

```

public static TabulatedFunction tabulate(Function f, double leftX, double rightX, int pointsCount) { 3 usages

    if (doubleLess(leftX, f.getLeftDomainBorder()) || doubleGreater(rightX, f.getRightDomainBorder())) {
        throw new IllegalArgumentException("Границы табулирования выходят за область определения функции");
    }

    if (pointsCount < 2) {
        throw new IllegalArgumentException("Количество точек табулирования должно быть не менее 2");
    }

    double[] values = new double[pointsCount];
    double step = (rightX - leftX) / (pointsCount - 1);

    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        values[i] = f.getFunctionValue(x);
    }

    return createTabulatedFunction(leftX, rightX, values);
}

```

```

public static TabulatedFunction inputTabulatedFunction(InputStream in) { no us
    try (DataInputStream dataIn = new DataInputStream(in)) {

        int count = dataIn.readInt();
        FunctionPoint[] points = new FunctionPoint[count];

        for (int i = 0; i < count; i++) {
            double x = dataIn.readDouble();
            double y = dataIn.readDouble();
            points[i] = new FunctionPoint(x, y);
        }

        return createTabulatedFunction(points);
    } catch (IOException e) {
        throw new RuntimeException("Ошибка ввода табулированной функции", e);
    }
}

```

```

public static TabulatedFunction readTabulatedFunction(Reader in) { no usages
    try {
        StreamTokenizer tokenizer = new StreamTokenizer(in);

        tokenizer.nextToken();
        int count = (int) tokenizer.nval;

        FunctionPoint[] points = new FunctionPoint[count];

        for (int i = 0; i < count; i++) {
            tokenizer.nextToken();
            double x = tokenizer.nval;

            tokenizer.nextToken();
            double y = tokenizer.nval;

            points[i] = new FunctionPoint(x, y);
        }

        return createTabulatedFunction(points);
    } catch (IOException e) {
        throw new RuntimeException("Ошибка чтения табулированной функции", e);
    }
}

```

Протестировала в main().

```

System.out.println("\n\nТест фабрик");
Function f = new Cos();

TabulatedFunction tf;
tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);
System.out.println("Фабрика по умолчанию: " + tf.getClass());

TabulatedFunctions.setTabulatedFunctionFactory(new
    ArrayTabulatedFunction.ArrayTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);
System.out.println("Array фабрика: " + tf.getClass());

TabulatedFunctions.setTabulatedFunctionFactory(new
    LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);
System.out.println("LinkedList фабрика: " + tf.getClass());

```

```

Тест фабрик
Фабрика по умолчанию: class functions.LinkedListTabulatedFunction
Array фабрика: class functions.ArrayTabulatedFunction
LinkedList фабрика: class functions.LinkedListTabulatedFunction

```

Задание 3

В классе `TabulatedFunctions` добавила ещё три перегруженные версии метода `createTabulatedFunction()`, получающие ссылку типа `Class` на описание класса, объект которого требуется создать.

```
//рефлексия
public static TabulatedFunction createTabulatedFunction(Class<?> clas, double leftX, double rightX, int pointsCount) {
    checkClass(clas);
    try {
        Constructor<?> constructor = clas.getConstructor(double.class, double.class, int.class);

        return (TabulatedFunction) constructor.newInstance(leftX, rightX, pointsCount);
    } catch (Exception e) {
        throw new IllegalArgumentException("При создании объекта через рефлексию произошла ошибка ", e);
    }
}

public static TabulatedFunction createTabulatedFunction(Class<?> clas, double leftX, double rightX, double[] values) {
    checkClass(clas);
    try {
        Constructor<?> constructor = clas.getConstructor(double.class, double.class, double[].class);

        return (TabulatedFunction) constructor.newInstance(leftX, rightX, values);
    } catch (Exception e) {
        throw new IllegalArgumentException("При создании объекта через рефлексию произошла ошибка ", e);
    }
}

public static TabulatedFunction createTabulatedFunction(Class<?> clas, FunctionPoint[] points) { 3 usages
    checkClass(clas);
    try {
        Constructor<?> constructor = clas.getConstructor(FunctionPoint[].class);

        return (TabulatedFunction) constructor.newInstance((Object) points);
    } catch (Exception e) {
        throw new IllegalArgumentException("При создании объекта через рефлексию произошла ошибка ", e);
    }
}

private static void checkClass(Class<?> clas) { 6 usages
    if (!TabulatedFunction.class.isAssignableFrom(clas)) {
        throw new IllegalArgumentException("Класс должен реализовывать TabulatedFunction");
    }
}
```

В классе `TabulatedFunctions` перегрузила методы `tabulate`, `inputTabulatedFunction`, `readTabulatedFunction`, создающие объекты табулированных функций, добавив версии, принимающие также ссылку типа `Class` на описание класса, объект которого требуется создать.


```

public static TabulatedFunction tabulate(Class<?> clas, Function f, double leftX, 1 usage
                                         double rightX, int pointsCount) {

    checkClass(clas);

    if (doubleLess(leftX, f.getLeftDomainBorder()) || doubleGreater(rightX, f.getRightDomainBorder())) {
        throw new IllegalArgumentException("Границы табулирования выходят за область определения функции");
    }

    if (pointsCount < 2) {
        throw new IllegalArgumentException("Количество точек табулирования должно быть не менее 2");
    }

    double[] values = new double[pointsCount];
    double step = (rightX - leftX) / (pointsCount - 1);

    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        values[i] = f.getFunctionValue(x);
    }

    return createTabulatedFunction(clas, leftX, rightX, values);
}

```

```

public static TabulatedFunction inputTabulatedFunction(Class<?> clas, InputStream in) {

    checkClass(clas);

    try (DataInputStream dataIn = new DataInputStream(in)) {

        int count = dataIn.readInt();
        FunctionPoint[] points = new FunctionPoint[count];

        for (int i = 0; i < count; i++) {
            double x = dataIn.readDouble();
            double y = dataIn.readDouble();
            points[i] = new FunctionPoint(x, y);
        }

        return createTabulatedFunction(clas, points);

    } catch (IOException e) {
        throw new RuntimeException("Ошибка ввода табулированной функции", e);
    }
}

```

```

public static TabulatedFunction readTabulatedFunction(Class<?> clas, Reader in) {

    checkClass(clas);

    try {
        StreamTokenizer tokenizer = new StreamTokenizer(in);

        tokenizer.nextToken();
        int count = (int) tokenizer.nval;

        FunctionPoint[] points = new FunctionPoint[count];

        for (int i = 0; i < count; i++) {
            tokenizer.nextToken();
            double x = tokenizer.nval;

            tokenizer.nextToken();
            double y = tokenizer.nval;

            points[i] = new FunctionPoint(x, y);
        }

        return createTabulatedFunction(clas, points);
    } catch (IOException e) {
        throw new RuntimeException("Ошибка чтения табулированной функции", e);
    }
}

```

Проверила в main().

```

System.out.println("\n\nТест рефлексии");
TabulatedFunction f1;

f1 = TabulatedFunctions.createTabulatedFunction(
    ArrayTabulatedFunction.class, leftX: 0, rightX: 10, pointsCount: 3);
System.out.println(f1.getClass());
System.out.println(f1);

f1 = TabulatedFunctions.createTabulatedFunction(
    ArrayTabulatedFunction.class, leftX: 0, rightX: 10, new double[] {0, 10});
System.out.println(f1.getClass());
System.out.println(f1);

f1 = TabulatedFunctions.createTabulatedFunction(LinkedListTabulatedFunction.class,
    new FunctionPoint[] {new FunctionPoint(x: 0, y: 10), new FunctionPoint(x: 10, y: 10)});
System.out.println(f1.getClass());
System.out.println(f1);

f1 = TabulatedFunctions.tabulate(LinkedListTabulatedFunction.class,
    new Sin(), leftX: 0, Math.PI, pointsCount: 11);
System.out.println(f1.getClass());
System.out.println(f1);

```

Тест рефлексии

class functions.ArrayTabulatedFunction

{(0,00; 0,00), (5,00; 0,00), (10,00; 0,00)}

class functions.ArrayTabulatedFunction

{(0,00; 0,00), (10,00; 10,00)}

class functions.LinkedListTabulatedFunction

{(0,00; 10,00), (10,00; 10,00)}

class functions.LinkedListTabulatedFunction

{(0,00; 0,00), (0,31; 0,31), (0,63; 0,59), (0,94; 0,81), (1,26; 0,95), (1,57; 1,00), (1,88; 0,95), (2,20; 0,81), (2,51; 0,59), (2,83; 0,31), (3,14; 0,00)}